```
from google.colab import files
uploaded = files.upload()
```

Choose Files customer_data.csv
customer_data.csv(text/csv) - 9676 bytes, last modified: 10/8/2025 - 100% done
Saving customer_data.csv to customer_data (1).csv

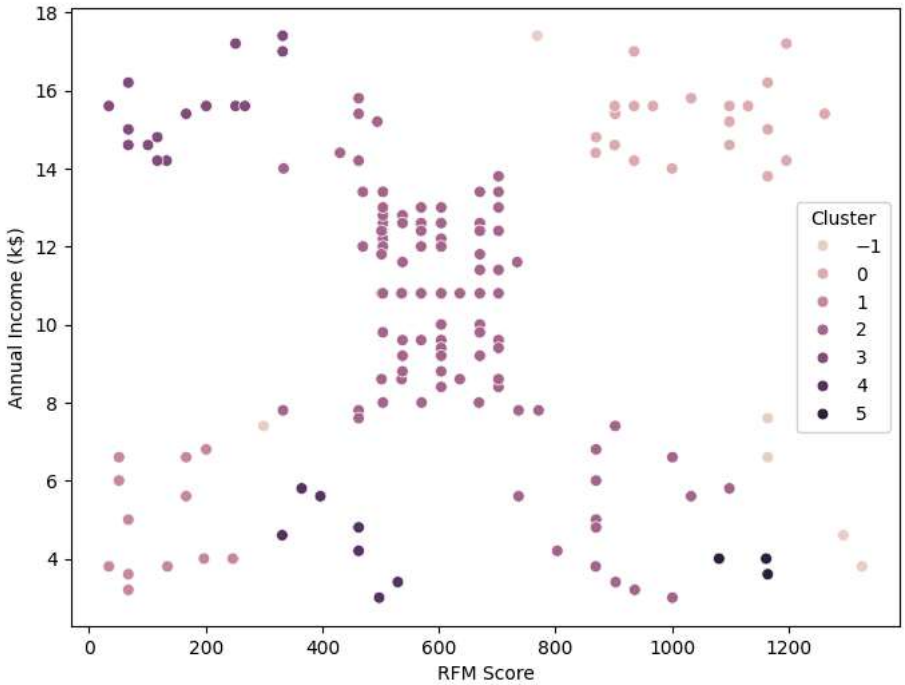
```
!pip install pandas numpy scikit-learn matplotlib seaborn
```

```
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: seaborn in /usr/local/lib/python3.12/dist-packages (0.13.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.3)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (26.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17.0
```

```
import pandas as pd
import numpy as np
import pickle
import os
from matplotlib import pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import DBSCAN
from sklearn.metrics import silhouette_score, davies_bouldin_score, calinski_harabasz_score

# Load dataset (IMPORTANT change for Colab)
df = pd.read_csv("customer_data.csv")

# OPTIONAL: speed up (remove later)
df = df.sample(min(1000, len(df)), random_state=42)

# Drop columns
df.drop(columns=['CustomerID','Gender','Age','Spending Score (1-100)','Mail id'], inplace=True)

# Weights
WEIGHT_RECENCY = 0.2
WEIGHT_FREQUENCY = 0.4
WEIGHT_MONETARY = 0.4
WEIGHT_INCOME = 0.2
WEIGHT_RFM = 0.8

# RFM
df['RFM Score'] = (
    WEIGHT_RECENCY * (100 - df['Recency (days)']) +
    WEIGHT_FREQUENCY * df['Frequency (visits)'] +
    WEIGHT_MONETARY * df['Monetary ($)']
)

x = df[['RFM Score','Annual Income (k$)']].copy()
x['RFM Score'] *= WEIGHT_RFM
x['Annual Income (k$)'] *= WEIGHT_INCOME

# Scaling
scaler = StandardScaler()
x_scaled = scaler.fit_transform(x)

# DBSCAN
db = DBSCAN(eps=0.45, min_samples=5, n_jobs=-1)
labels = db.fit_predict(x_scaled)

x['Cluster'] = labels
```

```
# Info
n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
noise = list(labels).count(-1)

print("Clusters:", n_clusters)
print("Noise points:", noise)

# Evaluation
mask = labels != -1

if n_clusters > 1:
    print("\nSilhouette:", round(silhouette_score(x_scaled[mask], labels[mask]),3))
    print("DB Index:", round(davies_bouldin_score(x_scaled[mask], labels[mask]),3))
    print("CH Score:", round(calinski_harabasz_score(x_scaled[mask], labels[mask]),3))

print("Noise Ratio:", round(noise/len(labels),3))

# Plot
plt.figure(figsize=(8,6))
sns.scatterplot(x=x['RFM Score'], y=x['Annual Income (k$)'], hue=x['Cluster'], s=40)
plt.show()
```

```
Clusters: 6
Noise points: 6

Silhouette: 0.392
DB Index: 0.577
CH Score: 87.316
Noise Ratio: 0.035
```




```
# %%
import pandas as pd

print("\n========== DBSCAN EVALUATION SUMMARY ==========")

mask = labels != -1

# Initialize values
sil_score = None
db_score = None

if n_clusters > 1:
    sil_score = silhouette_score(x_scaled[mask], labels[mask])
    db_score = davies_bouldin_score(x_scaled[mask], labels[mask])

noise_ratio = noise_points / len(labels)

# Create table
results = pd.DataFrame({
```

```
        "Metric": [
            "Silhouette Score",
            "Davies-Bouldin Index",
            "Noise Ratio"
        ],
        "Value": [
            round(sil_score, 3) if sil_score else "N/A",
            round(db_score, 3) if db_score else "N/A",
            round(noise_ratio, 3)
        ],
        "Quality": [
            "Good" if sil_score and sil_score > 0.5 else "Good" if sil_score and sil_score > 0.25 else "Poor",
            "Good" if db_score and db_score < 1 else "Good" if db_score and db_score < 2 else "Poor",
            "Good" if noise_ratio < 0.2 else "Average" if noise_ratio < 0.4 else "Poor"
        ]
    })

    print(results)
```

```
========== DBSCAN EVALUATION SUMMARY ==========
                 Metric  Value Quality
0      Silhouette Score  0.392    Good
1  Davies-Bouldin Index  0.577    Good
2           Noise Ratio  0.000    Good
```